Advanced Programming Tutorial
Worksheet 10: Performance modeling

Keywords: roofline model, peak performance, computational intensity, memory bandwidth

Prepare!

Before coming to the tutorial, and so that the time is enough to solve the exercise, revise the slides of the lecture unit "Performance modeling" and answer the following questions:

- ☐ Apart from solving a task faster, in which other aspects does performance help? → make use of resources → save costs and energy → Real-time apps
- ☐ What are the main characteristics of a computer that affect its performance? You may need to also revise the lecture unit "von Neumann architecture". → CPU Performance → Memory bandwidth
- ☐ What can the roofline model help us understand? What is its formula?
- ☐ What is pipelining and where does it help? → Hide latency of instructions
- ☐ What is the cache memory and how does it work? → Small chunk of memory close to the CPU → A lot faster than main memory
- ☐ Revise the lecture unit "Data types". What are the sizes of common data types? → Double: 8 Byte

Once you have revised this concepts, read the exercises that follow and think which of the above ingredients you will need. You don't need to solve the exercise at home, we will allocate approximately 30 min to try it during the tutorial, together with your neighbor.

Exercise 1: Roofline model

Since following the tutorials online has already become difficult, you take action and try to simulate vaccination models as fast as possible. For now, you start with a given implementation and your home computer. If you could invest on one resource to speed up your computations, what should this be: buy a faster CPU or buy faster memory?

The system you are using has a single core CPU with 2 GHz frequency ($2 \cdot 10^9$ *cycle / s*) and the following specifications:

- This CPU can perform 4 double-precision floating point operations simultaneously (i.e., it is vectorized, as we will see later) (*"double precision" means 8 bytes*)
- The latency of basic arithmetic operations `+`, `-`, `*` is 2 cycles *(one operation per two cycles)*.
- The latency of the fused multiply add (FMA) operation (`a += b * c`) is 3 cycles.
- These operations are **not** pipelined.
- This CPU has 2 load units and 1 store unit and each unit can accommodate 32 bytes (4 doubles)
- Accessing the cache memory **is pipelined** and costs 5 cycles. The cache operates at 2 GHz. (Hint: what does "is pipelined" mean for the cost?)

- Accessing the main memory **is not pipelined** and costs 10 cycles. The main memory operates at 800 MHz.
- Both types of memory can either load 2 vector registers (2×32 bytes) or store 1 vector register (32 bytes) simultaneously.

To decide whether we need to upgrade our CPU or main memory for running our simulation, we need to decide whether our simulation is memory-bound or compute-bound. We can do this by drawing the roofline model for the simulation.

For this, in general, we only need to compute three parameters: the peak floating point performance offered by our CPU, the rate at which data can be transferred from memory to CPU, i.e., the bandwidth, and the computational intensity of our code. However, here, our analysis for each of these parameters will break down into multiple cases: floating point operations can be either basic $(+, -, *)$, or be of the fused-multiply-add kind. We have 2 load units but only 1 store unit and pipelined access for cache but not for main memory; these lead to different bandwidths. We also have to consider these cases for computing the computational intensity for our simulation as well.

Let's determine the values of these parameters now. You may want to write the result next to each point.

1. First, we need to determine some performance properties of our floating point unit:
   (a) What is the theoretical maximum floating point performance for basic arithmetic operations?
   (b) What is the theoretical maximum floating point performance for fused multiply add operations?
2. Second, we have to determine some performance properties of our memory system (cache + main memory).
   (a) What is the theoretical read bandwidth of our memory? Only consider the cache and main memory by themselves without considering the connection to the CPU, i.e., do not consider the latency for memory operations here.
   (b) How much bandwidth can be saturated by our CPU when reading and when writing data? Assume that one load/store unit of the CPU can load/store one vector register. For this, you should consider the latency for memory operations.

   Remember that accessing the cache memory is pipelined, while accessing the main memory is not.
3. To help our next steps, draw the roofline model for this particular system.

4. Assume that the core computation in our simulation is matrix multiplication. Here the inner-most loop over i computes the (row,col) element of matrix C by taking the dot product of the row-th row of A with the col-th column of B, i.e., $C_{\text{row,col}} = A^T_{\text{row}} \cdot B_{\text{col}}$:

```
// C[row,column] = dot_product(A[row, :], B[:, col] )
// All the arrays are accessed as 1D, using pointer arithmetic
void mm_mult(size_t rows, size_t columns, size_t k,
             double *A, double *B, double *C) {
1 for (auto row = 0; row < rows; ++row) {
2  for (auto col = 0; col < columns; ++col) {
3   for (auto i = 0; i < k; ++i) {
        if (i == 0) {
          C[row*columns + col]  = A[row*k + i] * B[i*columns + col];
        } else {
          C[row*columns + col] += A[row*k + i] * B[i*columns + col];
        }
      }
    }
  }
}
```

(a) How many floating point operations are necessary for our computation?

(b) How many values need to be transferred from/to memory in the worst case (e.g., no optimizations)? Remember that `a += b * c` means `a = a + b * c`. Count all *memory interactions* without assuming that data is either in main memory or in cache.

(c) What is the computational intensity of this algorithm (both for reading and writing)?

(d) What performance can we expect from this algorithm when the cache is not involved? What is the limiting component in our system?

(e) What performance can we expect from this algorithm when all matrices are available in cache? What is the limiting component in our system?

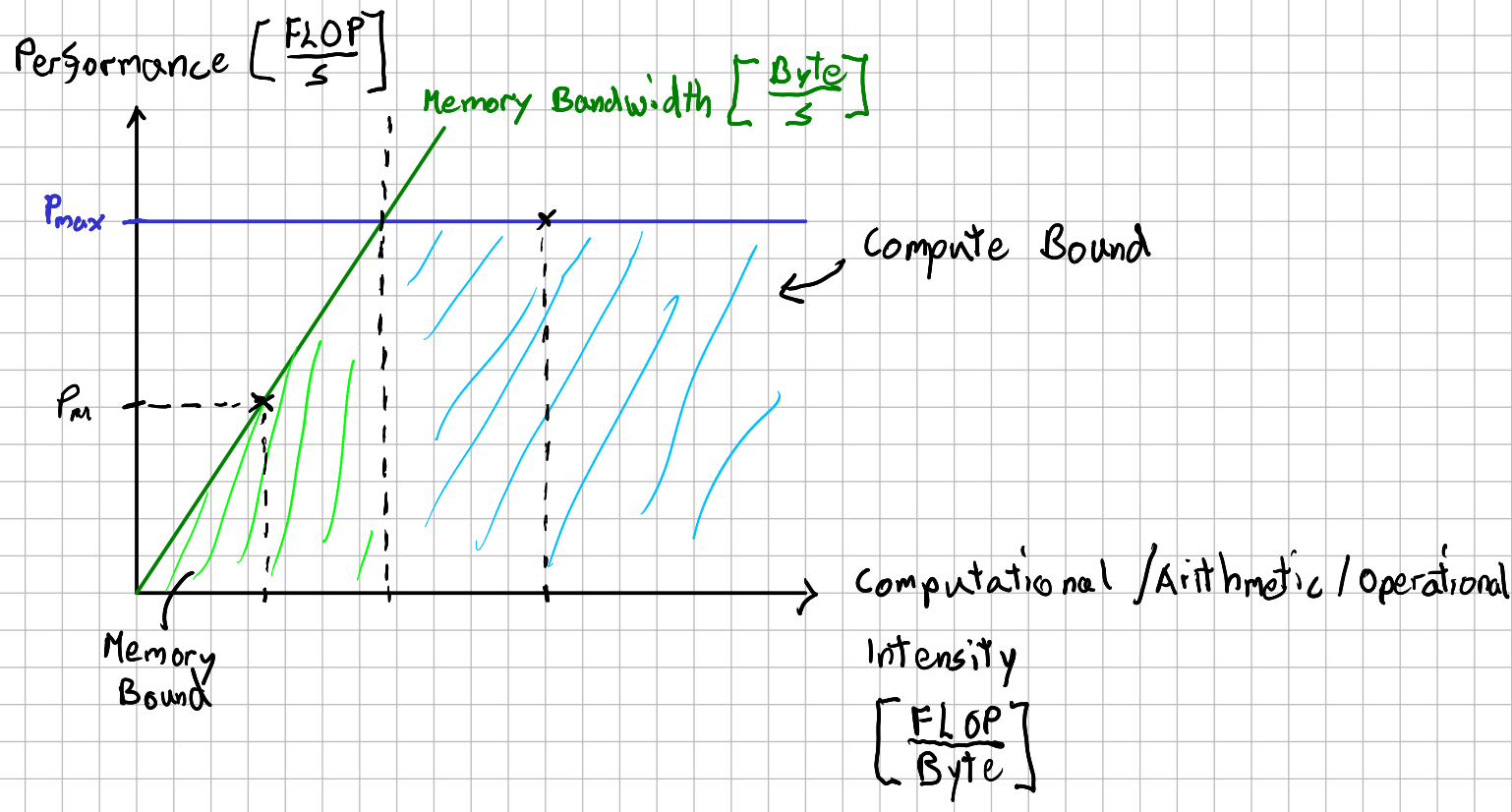
Congratulations, you won a discount for a brand-new, faster CPU! Do you want it?

Epilogue

In practice, most problems we deal with in simulations and similar domains are memory-bound, that's why we invest so much effort in efficiently using caches and minimizing memory interactions. Still, the roofline model is still useful when exploring optimization potential. Add a roofline analysis to your next HPC project proposal and impress your peers with your preparation!
Note: This exercise is way too long and complicated for the exam, but it fits the tutorial nicely, as it covers many aspects to discuss. See also the accompanying self-practice exercise.

Roofline Model

FLOP ≙ "Floating Point Operation"



Exercise

1

a) $\frac{4\ FLOP}{2\ cycles} \cdot 2\ \frac{Gcycles}{s} = 4\ \frac{GFLOP}{s}$

FMA ≙ $\underline{a} \mathrel{+}= \underline{b} \cdot \underline{c}$

b) $\frac{2 \cdot 4\ FLOP}{3\ cycles} \cdot 2\ \frac{Gcycles}{s} = \frac{16}{3}\ \frac{GFLOP}{s} \approx 5{,}33\ \frac{GFLOP}{s}$

2

a)

Main Memory: $\frac{2\ load\ unit}{cycle} \cdot \frac{32\ Byte}{load\ unit} \cdot 0{,}8\ \frac{Gcycles}{s} = 51{,}2\ \frac{GByte}{s}$

Cache: $\frac{2\ load\ unit}{cycle} \cdot \frac{32\ Byte}{load\ unit} \cdot 2\ \frac{Gcycles}{s} = 128\ \frac{GByte}{s}$

b) Memory:

Assume 10 CPU cycles latency

read: $\frac{2\ load\ units}{10\ cycles} \cdot \frac{32\ Byte}{load\ unit} \cdot 2\ \frac{Gcycles}{s} = 12{,}8\ \frac{GByte}{s}$

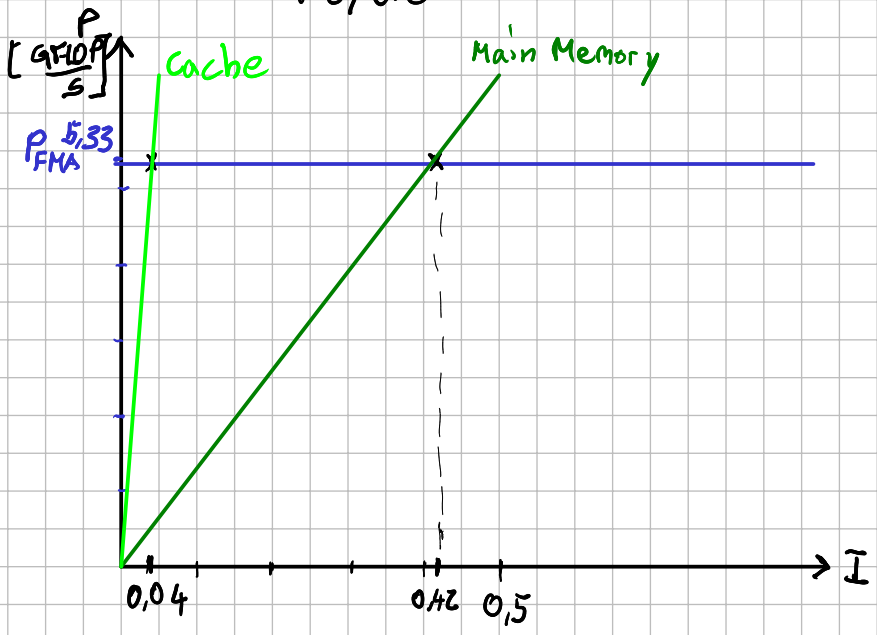
write: $\frac{1\ load\ unit}{10\ cycles} \cdot$ — " — $\cdot$ — " — $= 6{,}4\ \frac{GByte}{s}$

Cache:

read: $\frac{2 \text{ load units}}{1 \text{ cycle}} \cdot \frac{32 \text{ Byte}}{\text{load unit}} \cdot 2 \frac{\text{Gcycles}}{s} = 128 \frac{\text{GByte}}{s}$

Pipelining! → Disregard latency

write: $\frac{1 \text{ load unit}}{1 \text{ cycle}} \cdot$ — " — $\cdot$ — " — $= 64 \frac{\text{GByte}}{s}$



$I_M = \frac{P_{Max}}{b_S} = \frac{5{,}33 \text{ GFLOP/s}}{12{,}8 \text{ GByte/s}}$

$\approx 0{,}42 \frac{\text{FLOP}}{\text{Byte}}$

$I_C = \frac{P_{Max}}{b_S} = \frac{5{,}33 \text{ GFLOP/s}}{128 \text{ GByte/s}}$

$\approx 0{,}04 \frac{\text{FLOP}}{\text{Byte}}$

4)

a) loop 1: # rows

loop 2: # columns

loop 3: k

if(i==0): 1 FLOP (1 time)

else : 2 FLOP (k−1 times)

Total FLOP: $\text{rows} \cdot \text{columns} \cdot (1 \cdot 1 + 2 \cdot (k-1))$

$= \text{rows} \cdot \text{columns} \cdot (2k-1)$

b)

read A: $k \cdot c \cdot r$

read B: $k \cdot c \cdot r$

read C: $(k-1) \cdot c \cdot r$

Total reads: $(3k-1) \cdot c \cdot r$

write C: $k \cdot c \cdot r$

Total "Memory Interactions": $(4k-1) \cdot c \cdot r$

c) $I = \frac{FLOP}{Byte}$

$$I_{read} = \frac{(2k-1)\cdot c\cdot r \overbrace{FLOP}^{\text{all FLOP}}}{\underbrace{(3k-1)\cdot c\cdot r}_{\#\text{reads}} \cdot \underbrace{8\,Byte}_{\text{size per read}}} = \frac{2k-1}{24k-8} \overset{k\to\infty}{=} \frac{1}{12}\frac{FLOP}{Byte}$$

$$I_{write} = \frac{(2k-1)\cdot c\cdot r\ FLOP}{k\cdot c\cdot r\cdot 8Byte} = \frac{2k-1}{8k} \overset{k\to\infty}{=} \frac{1}{4}\frac{FLOP}{Byte}$$

d) Memory

$$\left.\begin{aligned} P_{read} &= I_{read}\cdot 12{,}8\,\frac{GByte}{s} \approx 1{,}066\,\frac{GFLOP}{s} \\ P_{write} &= I_{write}\cdot 6{,}4\,\frac{GByte}{s} \approx 1{,}6\,\frac{GFLOP}{s} \end{aligned}\right\} \text{Min: } 1{,}066$$

Limit is reading $P_{read} = 1{,}066 < P_{CPU} = 5{,}33$

↳ Memory Bound

e) Cache

$$P_{read} = I_r \cdot 128\,\frac{GByte}{s} = 10{,}66\,\frac{GFLOP}{s}$$

$$P_{write} = I_w \cdot 64\,\frac{GByte}{s} = 16\,\frac{GFLOP}{s}$$

Limit is reading $P_{read} = 10{,}66 > P_{CPU} = 5{,}33$

↳ Compute Bound